迷毂前端开发手册

一、快速开始

1. 项目地址

仓库地址:https://gitee.com/Fairyland_1/frcs.git 默认master分支。

2. 项目启动

> 需要安装nodejs、pnpm，建议版本nodejs>=18.17.0，pnpm保持最新
>
> nodejs安装地址：https://nodejs.org/en
>
> pnpm安装：npm install -g pnpm

> 首先检查是否安装了依赖node_models，如果没安装请先执行 pnpm install

pnpm run dev

3.项目打包

pnpm build

4. 配置部署

vite.config.ts文件：

```
server: {
      // 端口号
      port: VITE_PORT,
      host: "0.0.0.0",
      // 本地跨域代理 https://cn.vitejs.dev/config/server-options.html#server-proxy
      proxy: {
        "/api": {
          // api请求，这里填写后端地址
          target: "http://192.168.0.120:10101/api/v1",
          changeOrigin: true,
          rewrite: path => path.replace(/^\/api/, "")
        },
        "/messageHub": {
          // websocket请求这里填写后端地址
          target: "http://192.168.0.120:10101",
          ws: true, //代理websockets
          changeOrigin: true,
          rewrite: path => path.replace(/^\/messageHub/, "")
        },
        "/iconUrl": {
          // icon地址，icon服务器地址
          target: "http://192.168.0.120:8084",
          changeOrigin: true,
```

```
          rewrite: path => path.replace(/^\/iconUrl/, "")
        }
      },
      // 预热文件以提前转换和缓存结果，降低启动期间的初始页面加载时长并防止转换瀑布
      warmup: {
        clientFiles: ["./index.html", "./src/{views,components}/*"]
      }
    },
```

> - Signal的转发必须使用明确的ip，否则会报错

5. 文件夹配置

目录结构详见：[目录结构]

页面结构如下：

```
├── .husky  # 代码提交前校验配置文件
├── .vscode  # IDE 工具推荐配置文件
│   │   ├── extensions.json  # 一键安装平台推荐的 vscode 插件
│   │   ├── settings.json  # 设置扩展程序或 vscode 编辑器的一些属性
│   │   ├── vue3.0.code-snippets  # vue3.0 代码片段
│   │   └── vue3.2.code-snippets  # vue3.2 代码片段
│   │   └── vue3.3.code-snippets  # vue3.3 代码片段
├── build  # 构建工具
│   │   ├── cdn.ts  # 打包时采用 cdn 模式
│   │   ├── compress.ts  # 打包时启用 gzip 压缩或 brotli 压缩
│   │   ├── info.ts  # 输出打包信息（大小、用时）
│   │   ├── optimize.ts  # vite 依赖预构建配置项
│   │   ├── plugins.ts  # vite 相关插件存放处
│   │   ├── utils.ts  # 构建工具常用方法抽离
├── locales  # 国际化文件存放处
│   │   ├── en.yaml  # 英文配置
│   │   ├── zh-CN.yaml  # 中文配置
├── mock  # mock 模拟后台数据
│   │   ├── asyncRoutes.ts  # 模拟后台返回动态路由
│   │   ├── ...
├── node_modules  # 模块依赖
├── public  # 静态资源
│   │   ├── audio  # 音频文件
│   │   ├── html  # 静态 iframe 页面
│   │   ├── wasm  # wasm 文件以及胶水代码
│   │   ├── favicon.ico  # favicon
│   │   ├── logo.svg  # logo
│   │   ├── platform-config.json  # 全局配置文件（打包后修改也可生效）
├── src
│   ├── api  # 接口请求统一管理
│   ├── assets  # 字体、图片等静态资源
│   ├── components  # 自定义通用组件
│   │   ├── ReAnimateSelector  # [animate.css](https://animate.style/) 选择器组件
│   │   ├── ReAuth  # 按钮级别权限组件（根据路由meta中的auths字段进行判断）
```

```
│   │   ├── ReBarcode  # 条形码组件
│   │   ├── ReCol  # 封装 element-plus 的 el-col 组件
│   │   ├── ReCountTo  # 数字动画组件
│   │   ├── ReCropper  # 图片裁剪组件
│   │   ├── ReCropperPreview  # 图片裁剪预览组件
│   │   ├── ReDialog  # 基于 element-plus 中 el-dialog 组件开发的函数式弹框
│   │   ├── ReFlicker  # 圆点、方形闪烁动画组件
│   │   ├── ReFlop  # 时间翻牌组件
│   │   ├── ReFlowChart  # LogicFlow 流程图组件
│   │   ├── ReIcon  # 图标组件
│   │   ├── ReImageVerify  # 图形验证码组件
│   │   ├── ReMap  # 高德地图组件
│   │   ├── RePerms  # 按钮级别权限组件（根据登录接口返回的permissions字段进行判断）
│   │   ├── RePureTableBar  # 配合 `@pureadmin/table` 实现快速便捷的表格操作
https://github.com/pure-admin/pure-admin-table */
│   │   ├── ReQrcode  # 二维码组件
│   │   ├── ReSeamlessScroll  # 无缝滚动组件
│   │   ├── ReSegmented  # 分段控制器组件
│   │   ├── ReSelector  # 选择器组件
│   │   ├── ReSplitPane  # 切割面板组件
│   │   ├── ReText  # 支持 Tooltip 提示的文本省略组件
│   │   ├── ReTreeLine  # 树形连接线组件（基于element-plus）
│   │   ├── ReTypeit  # 打字机效果组件
│   │   ├── ReVxeTableBar  # 配合 vxe-table 实现快速便捷的表格操作
│   ├── config  # 获取平台动态全局配置
│   ├── directives  # 自定义指令
│   │   ├── auth  # 按钮级别权限指令（根据路由meta中的auths字段进行判断）
│   │   ├── copy  # 文本复制指令（默认双击复制）
│   │   ├── longpress  # 长按指令
│   │   ├── optimize  # 防抖、节流指令
│   │   ├── perms  # 按钮级别权限指令（根据登录接口返回的permissions字段进行判断）
│   │   ├── ripple  # 水波纹效果指令
│   ├── layout  # 主要页面布局
│   ├── plugins  # 处理一些库或插件，导出更方便的 api
│   ├── router  # 路由配置
│   ├── store  # pinia 状态管理
│   ├── style  # 全局样式
│   │   ├── dark.scss  # 暗黑模式样式适配文件
│   │   ├── element-plus.scss  # 全局覆盖 element-plus 样式文件
│   │   ├── reset.scss  # 全局重置样式文件
│   │   ├── sidebar.scss  # layout 布局样式文件
│   │   ├── tailwind.css  # tailwindcss 自定义样式配置文件
│   │   ├── ...
│   ├── utils  # 全局工具方法
│   │   ├── http  # 封装 axios 文件
│   │   ├── localforage  # 二次封装 localforage (https://localforage.docschina.org/) 支持设置过期时
间，提供完整的类型提示
│   │   ├── progress  # 封装 nprogress
│   │   └── auth.ts  # 处理用户信息和 token 相关
│   │   └── chinaArea.ts  # 汉字转区域码
│   │   └── globalPolyfills.ts  # 解决项目可能因为安装某个依赖出现 `global is not defined` 报错
│   │   └── message.ts  # 消息提示函数
```

```
│   │   ├── mitt.ts  # 触发公共事件，类似 EventBus
│   │   ├── preventDefault.ts  # 阻止键盘F12、浏览器默认右键菜单、页面元素选中、图片默认可拖动的方法
│   │   ├── print.ts  # 打印函数
│   │   ├── propTypes.ts  # 二次封装 vue 的 propTypes
│   │   ├── responsive.ts  # 全局响应式 storage 配置
│   │   ├── sso.ts  # 前端单点登录逻辑处理
│   │   ├── tree.ts  # 树结构相关处理函数
│   ├── views  # 存放编写业务代码页面
│   ├── App.vue  # 入口页面
│   ├── main.ts  # 入口文件
├── types  # 全局 TS 类型配置
│   │   ├── directives.d.ts  # 全局自定义指令类型声明
│   │   ├── global-components.d.ts  # 自定义全局组件获得 Volar 提示（自定义的全局组件需要在这里声明下才能获得 Volar 类型提示哦）
│   │   ├── global.d.ts  # 全局类型声明，无需引入直接在 `.vue` 、`.ts` 、`.tsx` 文件使用即可获得类型提示
│   │   ├── index.d.ts  # 此文件跟同级目录的 global.d.ts 文件一样也是全局类型声明，只不过这里存放一些零散的全局类型，无需引入直接在 .vue 、.ts 、.tsx 文件使用即可获得类型提示
│   │   ├── router.d.ts  # 全局路由类型声明
│   │   ├── shims-tsx.d.ts  # 该文件是为了给 .tsx 文件提供类型支持，在编写时能正确识别语法
│   │   └── shims-vue.d.ts  # .vue、.scss 文件不是常规的文件类型，typescript 无法识别，所以我们需要通过下图的代码告诉 typescript 这些文件的类型，防止类型报错
├── .browserslistrc  # 配置目标浏览器的环境
├── .dockerignore  # 排除不需要上传到 docker 服务端的文件或目录
├── .editorconfig  # 编辑器读取文件格式及样式定义配置 https://editorconfig.org/
├── .env  # 全局环境变量配置（当 .env 文件与 .env.development、.env.production、.env.staging 这三个文件之一存在相同的配置 key 时，.env 优先级更低）
├── .env.development  # 开发环境变量配置
├── .env.production  # 生产环境变量配置
├── .env.staging  # 预发布环境变量配置
├── .gitattributes  # 自定义指定文件属性
├── .gitignore  # git 提交忽略文件
├── .gitpod.yml  # gitpod 部署配置
├── .lintstagedrc  # lint-staged 配置
├── .markdownlint.json  # markdown 格式检查配置
├── .npmrc  # npm 配置文件
├── .nvmrc  # 用于指定在使用 Node Version Manager（NVM）时要使用的特定 Node.js 版本
├── .prettierignore  # prettier 语法检查忽略文件
├── .prettierrc.js  # prettier 插件配置
├── .stylelintignore  # stylelint 语法检查忽略文件
├── CHANGELOG.en_US.md  # 版本更新日志（英文版）
├── CHANGELOG.md  # 版本更新日志（英文版）
├── CHANGELOG.zh_CN.md  # 版本更新日志（中文版）
├── Dockerfile  # 用来构建 docker 镜像
├── LICENSE  # 证书
├── README.en-US.md  # README（英文版）
├── README.md  # README
├── commitlint.config.js  # git 提交前检查配置
├── eslint.config.js  # eslint 语法检查配置
├── index.html  # html 主入口
├── package.json  # 依赖包管理以及命令配置
├── pnpm-lock.yaml  # 依赖包版本锁定文件
```

```
├── postcss.config.js  # postcss 插件配置
├── stylelint.config.js  # stylelint 配置
├── tailwind.config.ts  # tailwindcss 配置
├── tsconfig.json  # typescript 配置
└── vite.config.ts  # vite 配置
```

View下为页面文件
文件中

- index.vue为主页面
- Form为表单页面
- Hook.tsx 为主页面逻辑
- Rule.ts 为表单验证
- Type.ts 为类型验证

```
import { useHook } from "./utils/hook";
const {
  query,
  loading,
  columns,
  dataList,
  pagination,
  deviceDetection,
  handleSelection,
  handlePageSize,
  handlePageCurrent,
  handleSearch,
  handleReset,
  handleDetail,
  handleSite
} = useHook(tableRef);
```

- λ 使用useHook的方式传入表单ref或者表格ref，在tsx文件中调用
- λ Hooks中暴露一个useHook方法，里面写入数据与方法
- λ 表格组件使用PureTableBar，详情可以参考pure-admin-table官网

```
 addDialog({
      title: "站点",    //标题
      width: "60%",     //宽度
      alignCenter: true, //对齐方式
      draggable: true,   //能够拖动
      fullscreenIcon: true,  //全屏图标
      hideFooter: true,    //隐藏表单尾部
      props: { formInline: handleRow(row) },
      contentRenderer: () => siteIndex
});
```

- λ 弹框是使用的pure-admin函数式弹框
- λ contentRenderer使用h渲染函数，可以到addDialog的组件中查看详情
- λ props为传递到dialog当中的信息

二、地图信息交互

1.创建地图

```
const page: any = {
    timerId: null,
    canvas: {},
    panel: {},
    box: {},
    init: () => {
      page.box = document.getElementById("boxPanel");
      page.canvas = document.getElementById("canvas");
      page.panel = new Panel(page.box, page.canvas, {
        map: { base: { size: { w: 13000, h: 9000 } } },
        mode: "Read"
      }).init();
      page.bind();
    },
    bind: () => {
      page.panel.panelActiveAction = event => {
        carCode = "";
      };
      page.panel.nodeRectActiveEnterAction = event => {
        carCode = "";
        page.readSiteInfo(event, "站点");
      };
      page.panel.nodeEllipseActiveEnterAction = event => {
        carCode = "";
        page.readSiteInfo(event, "站点");
      };
      page.panel.edgeBeelineActiveEnterAction = event => {
        carCode = "";
        page.readSiteInfo(event, "路线");
      };
```

```
      page.panel.carActiveEnterAction = event => {
        carCode = event.code.text;
        //console.log(event);
        page.readSiteInfo(event, "车辆");
      };
      page.panel.edgeArcActiveEnterAction = event => {
        carCode = "";
        page.readSiteInfo(event, "路线");
      };
      page.panel.edgeQuadraticActiveEnterAction = event => {
        carCode = "";
        page.readSiteInfo(event, "路线");
      };
      page.panel.edgeBezierActiveEnterAction = event => {
        carCode = "";
        page.readSiteInfo(event, "路线");
      };
      // window.addEventListener("resize", event => page.panel.setResize());
},
```

- λ 使用init来初始化画布
- λ Bind来绑定方法，可以使用与fass中一样的方法，来进行覆盖

```
watch(
  () => tagEllipseForm,
  (newVal, oldVal) => {
    if (!isWatchChange.value) return;
    let oldObj = parse(cloneTagEllipseForm);
    const changes = findDifferences(newVal, oldObj, "", ["base.p1", "base.p2"]);
    changes.forEach(({ path, value }) => {
      page.panel.setActiveShapeParam(path, value, "TagEllipse");
    });
    cloneTagEllipseForm = stringify(newVal);
  },
  { deep: true } // 深度监听对象的属性变化
);
```

- λ 地图编辑的批量编辑使用的watch监听
- λ 与Fass监听所有input的方式不一样，vue是虚拟dom，使用那个比较麻烦，这里使用watch的方式，使用字段监听，来进行批量编辑

2.使用地图

1.建立连接

使用signal库来使用websocket，在useMessageHub.ts中

```
export { connect, send, disconnect, setupSignalRHandlers };
```

导出4个方法：

- connect创建连接
- Send 发送数据
- Disconnect 关闭连接
- setupSignalRHandlers 获取数据

> 需要手动先发一次update信息，后端捕获到 update之后才会发送信息

> 需要在setupSignalRHandlers之后再次发送update类似于递归实现信息的实时获取

2.转换地图

- 在transform.js中，拿到S中地图信息，转化为Fass地图主键信息格式

```
function findNodeIdByCode(nodes, track, startNode = true) {
  const code =
    track.direction !== "b2a"
      ? startNode
        ? track.a
        : track.b
      : startNode
        ? track.b
        : track.a;
  // const code = startNode ? track.a : track.b;
  const node = nodes.find(node => node.id === String(code));
  return node ? node.id : null;
}
```

- λ 获取S中路线的方向，转化为Fass中地图组件方向

```
  init: (Json, storages) => {
    let newJson = {};
    let mapJson = Json;
    mapTransform.changeMap(mapJson, newJson);
    mapTransform.changeSites(mapJson, newJson, storages);
    mapTransform.changeTracks(mapJson, newJson);
    mapTransform.changeCircularArcTracks(mapJson, newJson);
    mapTransform.changeBezierTracks(mapJson, newJson);
    console.log("newJson is", mapJson, newJson);
    return newJson;
  },
```

- λ 使用init方法转化原始数据
- λ changeMap设置地图信息，会有node的基类，根据基类来转换

```
const xValues = mapJson.sites.map(site => site.x);
    const yValues = mapJson.sites.map(site => -site.y); // 对y取负
    // 获取x的最小值和最大值
    const minX = Math.min(...xValues);
    const maxX = Math.max(...xValues);
    // 获取y取负后的最小值和最大值
    const minY = Math.min(...yValues);
    const maxY = Math.max(...yValues);
    AbsMinX = Math.abs(minX);
    AbsMaxX = Math.abs(maxX);
    AbsMinY = Math.abs(minY);
    AbsMaxY = Math.abs(maxY);
    MapPaddingLeft = (AbsMinX + AbsMaxX) * 0.1;
    MapPaddingTop = (AbsMinY + AbsMaxY) * 0.1;
    newJson.map.base.size.w =
      (AbsMinX + AbsMaxX) * 1.2 + 2 * normNode.base.size.w;
    newJson.map.base.size.h =
      (AbsMinY + AbsMaxY) * 1.2 + 2 * normNode.base.size.h;
    newJson.map.base.center.x = AbsMinX + AbsMaxX / 2;
    newJson.map.base.center.y = AbsMinY + AbsMaxY / 2;
    let AbsExtend = {
      AbsMinX: AbsMinX + MapPaddingLeft,
      AbsMaxX: AbsMaxX,
      AbsMinY: AbsMinY + MapPaddingTop,
      AbsMaxY: AbsMaxY,
      // normNodeW: normNode.base.size.w / 2,
      // normNodeH: normNode.base.size.h / 2
      normNodeW: 0,
      normNodeH: 0
    };
    newJson.map.extends[0] = {
      key: "Abs",
      value: JSON.stringify(AbsExtend)
};
```

- λ 为了让地图能够剧中展示，需要遍历获取站点的位置，拿到最大最小值
- λ 需要注意的是，s中的坐标位置是第四象限，Fass组件中地图默认坐标系是第一下象限，在这里获取最大最小值，然后1.2倍，两边各留10%的间隙

三、新增页面/业务

1. 新增一级菜单下页面

在views下新建目录，名称以后端返回路由为主

例如：新增一个**使用说明**菜单>**开发手册**页面，后端返回路由为：Instructions/Development/Index

- 页面部分

1. 在views下新建Instructions文件夹

2. 在Instructions创建Development文件夹
3. 在Development创建index.vue文件，index为当前页面的入口文件
4. 在index页面编写UI代码

```
<script setup lang='ts'>
  // 此处编写js相关代码。建议写到hook中去，便于维护
</script>

<template>

  <div>此处编写模版代码</div>
</template>

<style lang='scss' scoped>
// css样式相关
</style>
```

> 相关ui组件可参考 https://element-plus.org/zh-CN/component/overview.html

5. 如有逻辑代码请创建utils>hook.tsx文件,在该文件中写js逻辑代码

```
import { type Ref, h, ref, reactive, onMounted } from "vue";
export function useHook(tableRef: Ref) {
  // 这里写相关逻辑代码
  const columns: TableColumnList = [// 表格表头相关
    {
      headerRenderer: () => t("table.number"), // 表头名字，t("table.number")是国际化配置的
      label: "table.number", //筛选用的，也是使用国际化配置
      prop: "id", // 绑定字段
      hide: true // 是否隐藏，默认false
    },
  ] ,
    // 该函数是返回的getpag搜索需要的参数，根据需要调整；
   function handlePageParam() {
    return {
      pageParam: JSON.stringify({
        where: Object.keys(query)
          .filter(e => query[e])
          .map(e => ({
            logic: "And",
            field: e,
            operator: "Equal",
            value: query[e]
          })),
        order: [{ field: "createAt", sequence: "DESC" }],
        number: pagination.currentPage,
        size: pagination.pageSize
      })
    };
```

```
  }
  // 默认搜索函数
    async function handleSearch() {
      loading.value = true;  //开启加载状态
      pageParam.value = handlePageParam(); // 获取请求参数
      const { data } = await getPage(pageParam.value);//发送api请求
      dataList.value = data.rows; // 将请求结果赋值给对应变量
      pagination.total = data.total; // 页数
      loading.value = false; //关闭加载状态
    }
}
```

- api相关

1. 如有api请求相关的请在**src>api**目录下创建跟页面部分同样的文件夹>Instructions>Development
2. 创建index.ts文件

```
import { http } from "@/utils/http";
import { baseUrlApi } from "../../utils";
import type { ReponseResult } from "../../types";

export const getPage = (params?: object) => {
  return http.request<ReponseResult>("get", baseUrlApi("Base/Edge/GetPage"), {
    params
  });
};
```

3. 去页面引入使用即可 hook.tsx

```
import { getPage } from "@/api/Instructions/Development";// 引入api
```

```
  async function handleSearch() { // 使用api
    loading.value = true;
    pageParam.value = handlePageParam();
    console.log(pageParam.value);
    const { data } = await getPage(pageParam.value);
    dataList.value = data.rows;
    pagination.total = data.total;
    loading.value = false;
  }
```

2.新增二级菜单下页面

同1，直接根据返回路由找到对应文件夹，新建即可。

3.修改表单页面表头等数据

页面表头数据是在hook.tsx中columns中声明的：

```
const columns: TableColumnList = [
  { type: "selection", align: "left", sortable: true },
  {
    headerRenderer: () => t("table.number"),
    label: "table.number",
    prop: "id",
    hide: true
  },
  ...
  ]
```

headerRenderer:表头名称

label: 筛选名称

prop: 传值字段，跟接口数据字段保持一致

hide: true，是否隐藏该列

sortable: true，是否开启排序

formatter：可以在此方法中定义一些函数，返回值会覆盖props

4.修改表单页面搜索条件

在index中新建需要的标签，例如 新增一个创建时间的搜索条件：

```
    <el-form
      ref="formRef"
      :inline="true"
      :model="query"
      class="search-form bg-bg_color w-[99/100] pl-8 pt-[12px] overflow-auto"
    >
      <el-form-item :label="$t('table.createAt')" prop="time">
        <div class="block">
          <el-date-picker
            v-model="query.time"
            type="daterange"
            unlink-panels
            range-separator="To"
            start-placeholder="Start date"
            end-placeholder="End date"
            :shortcuts="shortcuts"
          />
        </div>
      </el-form-item>
```

在hook.tsx中的query新增：

```
const query = reactive({
  carId: "",
  actionType: "",
  state: "",
  time: [dayjs().subtract(1, "week").toDate(), new Date()] // 新增的字段，里面是默认值
});
```

5.新增表单页面按钮

在index.vue文件中PureTableBar组件里新增：

```
<el-button
           type="primary"
           :icon="useRenderIcon(Plus)"
           @click="handleAdd()"
           >
  {{ $t("buttons.add") }}
</el-button>
```

> type:按钮类型
>
> :icon：按钮图标
>
> @click：点击按钮回调函数
>
> {{ $t("buttons.add") }}：国际化方式

此后在hook.tsx中新增一个同名方法，之后导出，在index中引入即可

hook：

```
async function handleAdd() {
  pageParam.value = handkleCarPageParam();
  const { data } = await GetCarListToSelect();
  carList.value = data;
}
......
......
return{
....
handleAdd,
....
}
```

index.vue:

```
const {
  ....
  handleAdd,
  ....
} = useHook(tableRef);
```

6.弹窗页面、数据传递

使用addDialog组件，需要传入一个对应的组件

例如添加，在hook中：

```
 function handleAdd() {
    addDialog({
      title: "添加",  // 弹窗表头
      width: "60%", //弹窗宽度
      alignCenter: true,//居中对齐
      draggable: true,//是否可拖拽
      fullscreenIcon: true,//是否可以全屏
      closeOnClickModal: false,//点击关闭
      props: { //props传值，将hook中数据传递到表单页面
        formInline: handleRow()//对应数据
      },
      contentRenderer: () => h(editForm, { ref: formRef }),//editForm要加载的组件，formRef要拿的数据
      beforeSure: (done, { options, index }) => { //点击确定/取消按钮进行的一系列操作
        const formData = options.props.formInline as FormItemProps;
        formData.state = "Created";
        formRef.value.getRef().validate(async valid => {
          if (valid) {
            await addOrUpdate("", formData);
            message("操作成功！", { type: "success" });
            handleSearch();
            done();
          }
        });
      }
    });
  }
```

对应组件中：

```
const props = withDefaults(defineProps<FormProps>(), {
  formInline: () => ({
    carId: "",
    actionType: "",
    actionDescription: "",
    blockingType: "",
    remark: ""
  })
});//收到hook中的传值信息
```

```
const newFormInline = ref(props.formInline);


function getRef() {
  return ruleFormRef.value;//定义一个回传的方法，主要是把表单数据传递到hook中
}

defineExpose({ getRef });//暴露此方法，交给hook调用
```

四、库位任务可视化

panel文件夹

`hook.tsx` 文件是 `visualizationtask` 项目中用于管理仓库可视化面板的核心逻辑文件。它主要负责处理数据的获取、状态管理、画布初始化和用户交互等功能。

核心功能

- **数据获取与处理**: 从后端获取仓库区域和存储数据，并进行处理以供前端展示。
- **画布初始化**: 使用 Konva.js 初始化画布，绘制仓库布局。
- **用户交互**: 处理用户的点击、拖动、缩放等交互操作。
- **状态管理**: 管理和更新仓库区域的状态信息。

数据处理

- `handlePageParam()`: 构建分页参数，用于请求数据。
- `handleGetList(row)`: 根据选定的区域获取数据列表，并初始化画布。

画布操作

- `canvasInit()`: 初始化 Konva 画布，绘制仓库布局。
- `setSketchpad()`: 设置画布的缩放和位置状态。
- `handleSave()`: 保存当前画布状态和数据到后端。

用户交互

- `handleEdit()`: 切换编辑模式，允许用户拖动和调整画布元素。
- `updateColor()`: 更新画布元素的颜色和文本信息。

使用说明

1. **初始化**: 在组件挂载时调用 `onMounted` 钩子，初始化数据和画布。
2. **数据获取**: 使用 `handleGetList` 函数获取并展示指定区域的数据。
3. **编辑模式**: 调用 `handleEdit` 切换编辑模式，允许用户调整画布布局。
4. **保存状态**: 调用 `handleSave` 将当前画布状态保存到后端。

注意事项

- **数据格式**: 确保从后端获取的数据格式正确，特别是坐标和状态信息。

- **性能优化**: 由于画布操作可能涉及大量 DOM 操作，建议在必要时使用 `batchDraw` 优化性能。
- **错误处理**: 在数据请求和保存时，添加必要的错误处理逻辑，确保用户体验。

menu文件夹

menu 文件夹主要负责管理仓库可视化任务中的菜单功能。菜单用于选择和切换不同的仓库区域，以便在可视化面板中展示相应的内容。

主要文件

1. index.vue:

这是菜单组件的主文件，负责渲染菜单界面和处理用户交互。

使用了 Element Plus 的 el-radio-group 和 el-radio 组件来实现区域选择。

通过 handleGetList 方法来获取和展示选定区域的数据。

关键功能

区域选择:

菜单提供了一个单选按钮组，用户可以选择“全部”或特定的仓库区域。

选择区域后，会调用 handleGetList 方法来更新可视化面板中的数据。

动态渲染:

菜单根据传入的 areaList 动态渲染区域选项。

如果 areaList 为空，则显示“无数据”提示。

组件交互

与父组件的交互:

通过 defineProps 接收 areaList 和 handleGetList 两个属性。

areaList 是一个包含所有可用区域的列表。

handleGetList 是一个方法，用于在选择区域后获取相应的数据。

样式和布局:

使用了 Element Plus 的布局组件 el-row 和 el-col 来实现响应式布局。

自定义了一些样式以适应项目的视觉设计。

注意事项

数据更新: 确保 areaList 数据在父组件中正确传递，并在区域变化时及时更新。

sendwoek文件夹概述

sendwork 文件夹主要负责管理仓库可视化任务中的任务下发功能。该功能允许用户通过界面选择任务模板、车辆类型、起点和终点等信息，并将任务下发到指定的车辆。

主要文件

\1. index.vue:

这是任务下发的主界面文件，负责渲染任务下发表单和处理用户交互。

使用了 Element Plus 的 el-form、el-select 和 el-button 组件来实现表单输入和提交。

通过 useHook 函数获取和管理表单数据及相关操作。

\2. select.vue:

该文件提供了一个选择界面，用于设置任务的起点和终点。

使用了图片和按钮组合的方式，用户可以通过点击按钮来设置起点、终点或呼叫库位。

utils/hook.tsx:

包含任务下发功能的核心逻辑。

提供了数据获取、表单验证、任务下发等功能。

使用 onMounted 钩子在组件挂载时初始化数据。

utils/rule.ts:

定义了表单的验证规则，确保用户输入的数据符合要求。

使用 element-plus 的 FormRules 类型来定义验证规则。

utils/types.ts:

定义了表单数据的类型接口，确保数据结构的一致性。

关键功能

任务下发:

用户可以选择任务模板、车辆类型、起点和终点等信息。

表单提交后，调用 handleSendWorkFn 函数将任务下发到后端。

表单验证:

使用 formRules 定义表单验证规则，确保用户输入的数据有效。

验证包括必填项检查和起点终点的重复检查。

数据获取:

使用 GetAreaListToSelect 和 GetCarPage 等 API 获取区域和车辆数据。

数据在组件挂载时通过 onMounted 钩子进行初始化。

组件交互

与状态管理的交互:

使用 Pinia 的 useFormStateStore 管理表单状态。

通过 watch 监听状态变化，动态更新表单数据。

与其他组件的交互:

通过 props 接收父组件传递的 handleSendWorkFns 方法，用于处理任务下发后的操作。

注意事项

数据同步: 确保在表单数据和状态管理之间保持同步，避免数据不一致。

错误处理: 在任务下发和数据获取时，添加必要的错误处理逻辑，确保用户体验。